

Support Agent Implementation Report

Building a vertical AI support agent with Google ADK that resolves 70%+ of customer tickets autonomously — so DataByt can run solo until revenue justifies a support hire.

Google ADK (TypeScript) · Gemini · pgvector · Supabase · Next.js | v1.0 · June 2026

0. The Strategy

Customer support is where most solo-founder SaaS dies. A CFO paying ₹39,000/month expects an answer in minutes, not "I'll get back to you." You cannot be awake 24/7 — an AI agent that knows DataByt deeply is your unfair advantage.

The bet: VERTICAL, not horizontal

Zendesk AI and Intercom Fin are *horizontal* — they know nothing about AR, dunning, QuickBooks OAuth, or your product. Your agent knows DataByt better than any human rep, because it is trained on your docs and your past tickets, and it can take real actions inside your system.

Horizontal support AI
"Let me find that for you"
generic · hallucinates
knows nothing about AR
can't take actions

Your vertical agent
"Your QuickBooks token expired 14 days
ago — I've flagged it, here's the
2-click reconnect link"
checks invoice status, resends emails,
reconnects integrations, then replies

The 70/30 rule

- **70% of tickets** are repetitive ("how do I connect QuickBooks", "where's my data", "an email didn't send") → agent auto-resolves.
- **30% need you** (billing disputes, bugs, angry customers) → agent escalates with full context so your reply takes 2 minutes, not 20.

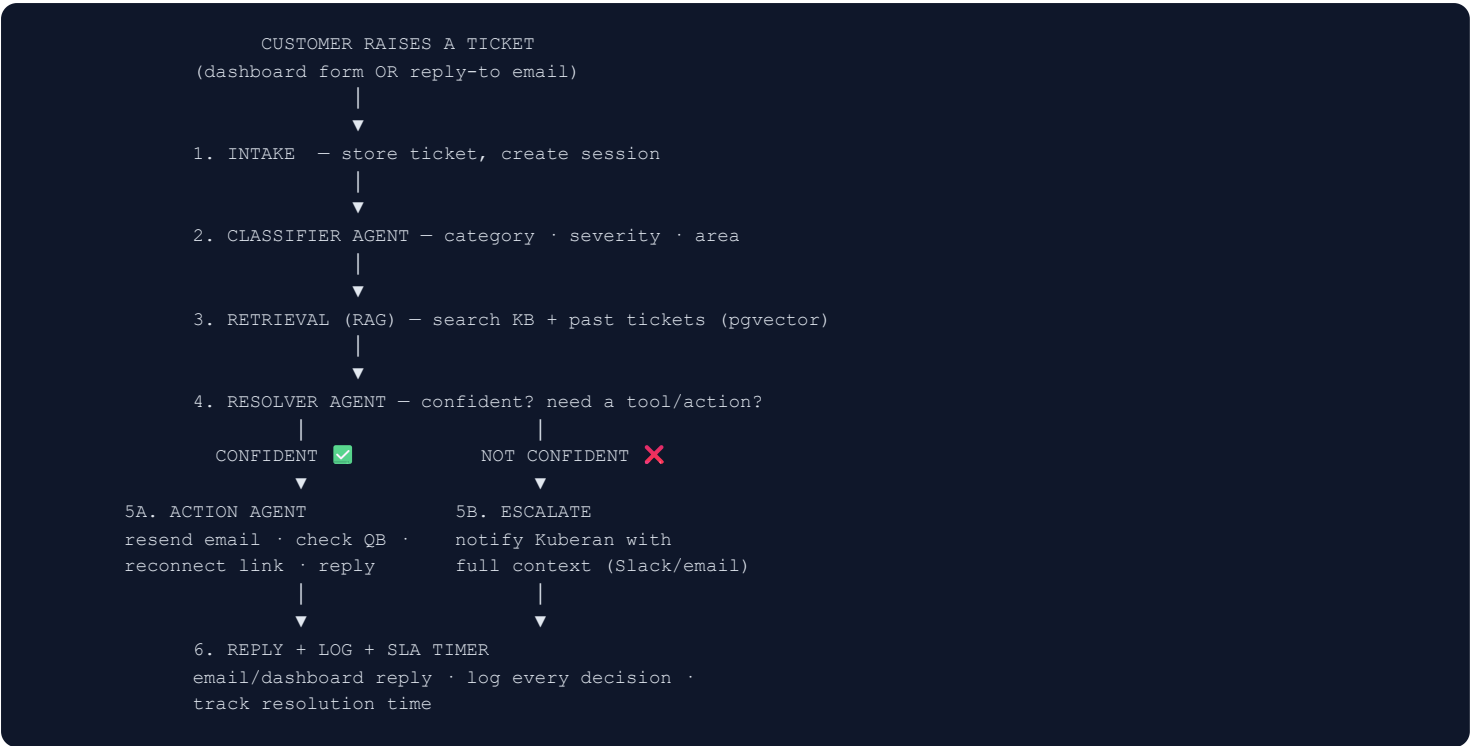
1. What is Google ADK?

ADK = Agent Development Kit — Google's open-source agent framework. It got official **TypeScript support (adk-js) in December 2025**. That matters: DataByt is TypeScript, so you keep ONE language across your whole stack.

Concept	What it is	DataByt example
Agent	LLM + instructions + tools	The "Support Agent"
Tool	A function the agent can call	<code>getInvoiceStatus()</code>
Sub-agent	Specialised agent it delegates to	Classifier, Resolver
Runner	Executes an agent turn	Runs when a ticket arrives
Session	One conversation thread	One support ticket
State	Memory within a session	"already tried reconnecting"
Memory	Knowledge across sessions	"this org always has QB issues"
Callbacks	Hooks before/after calls	Logging, guardrails, cost
Workflow agents	Sequential/Parallel/Loop	Classify → Retrieve → Resolve

Why ADK for support (LangGraph for collections): ADK has native Gemini/Vertex integration (cheapest on your credits), built-in tool-calling + sessions (less boilerplate), and Google's Agent Engine can host it later. Learning both ADK + LangGraph makes you employable in both ecosystems. You're learning *concepts* — agents, tools, RAG, memory, eval — which transfer everywhere.

2. The Architecture



How it slots into your stack

Existing	New (support agent)
Next.js + TypeScript	ADK agents run inside API routes
Supabase (Postgres)	+ support_tickets, ticket_messages, kb_documents
pgvector (in Supabase)	embeddings for RAG (KB + past tickets)
Gemini API	the LLM powering every agent
Resend	inbound = email→ticket · outbound = replies
Vercel cron	SLA breach checker

Key decision: the agent runs **inside your Next.js app** (in `/api/support/*` routes). No separate Python service, no Cloud Run. One deploy, one language, one bill.

3. Tools To Learn

A. Inside GCP

Tool	Why	Priority	Time
Google ADK (adk-js)	The agent framework	Critical	1 wk
Gemini API	Tool-calling, structured output	Critical	2 d
Vertex AI (basics)	Where Gemini scales; Agent Engine	Useful	2 d
GCP billing / credits	Manage 3-month runway	Critical	1 d
Cloud Logging	See agent behaviour in prod	Later	1 d

Skip for now (saves money): Vertex Vector Search (expensive — use pgvector), Agent Engine hosting (run in Next.js first), Document AI, Dialogflow (legacy).

B. Outside GCP (smart-money tools)

Tool	Why	Cost	Priority
pgvector (Supabase)	Agent memory + RAG, already in stack	Free	Critical
Langfuse (self-host)	Trace every decision, debug, track cost	~\$5 VPS	Critical
Resend Inbound	Email reply → ticket	Free tier	Critical
Trigger.dev / Vercel Cron	SLA checks, async work	Free tier	Useful
Zod	Validate agent output (in your deps)	Free	Useful

4. Implementation in the Codebase

New tables (Supabase)

```
-- Tickets
CREATE TABLE support_tickets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID NOT NULL REFERENCES organizations(id),
  subject TEXT NOT NULL,
  status TEXT DEFAULT 'open',      -- open|resolved|escalated|closed
  category TEXT, severity TEXT,
  resolved_by TEXT,               -- 'agent' | 'human'
  created_at TIMESTAMPTZ DEFAULT now(),
  resolved_at TIMESTAMPTZ, sla_due_at TIMESTAMPTZ
);

-- Knowledge base with embeddings
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE kb_documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT, content TEXT,
  embedding vector(768),         -- text-embedding-004
  source TEXT, created_at TIMESTAMPTZ DEFAULT now()
);
```

New folder structure

```
src/lib/support-agent/
├─ agents/   classifier.ts · resolver.ts · orchestrator.ts
├─ tools/    getInvoiceStatus · getIntegrationStatus ·
│             resendEmail · searchKnowledge · escalateToHuman
├─ memory/   pgvector.ts      (embed + retrieve)
├─ runner.ts
└─ types.ts

src/app/api/support/
├─ ticket/route.ts      POST: create ticket → run agent
├─ reply/route.ts       POST: customer replies → continue
└─ sla-check/route.ts   GET (cron): flag SLA breaches

src/app/dashboard/support/page.tsx  ticket queue UI
src/app/api/webhooks/support-inbound/route.ts  email → ticket
```

Your secret weapon — the seed step: embed your existing [DataByt-Guide/](#) docs into `kb_documents` . The agent's knowledge IS your documentation. Every doc you write makes the agent smarter, for free. Past resolved tickets get embedded too — so it compounds.

5. Build & Test

Build sequence (~3 weeks of focused work)

1	Tables + migration	1 d
2	Embed guide docs into kb_documents	1 d
3	searchKnowledge tool (pgvector RAG)	2 d
4	Classifier agent (category/severity)	2 d
5	Resolver agent (answer vs escalate)	3 d
6	Action tools (invoice, integration, resend)	3 d
7	escalateToHuman (Slack/email notify)	1 d
8	Support dashboard page	2 d
9	Resend inbound webhook (email→ticket)	2 d
10	SLA cron + Langfuse observability	2 d

How to test — the discipline that makes it reliable

1. **Golden test set** — 30 real CFO questions with expected behaviour:

```
"How do I connect QuickBooks?"      → auto-resolve, KB answer
"My invoices aren't syncing"        → check integration tool
"I want a refund"                   → escalate to human
"Change my dunning to 5 days"       → KB answer + settings link
"An email bounced, can you resend?" → resendEmail tool
```

2. Run the eval — for each: correct category? correct resolve-vs-escalate? right answer? right tool?

3. Drive it like a user — submit tickets through the form and reply-to email; watch in Langfuse. A blank reply or wrong tool call is a failure.

The escalation safety net: when confidence is low, it MUST escalate — never guess. A wrong confident answer to a CFO is worse than an honest escalation. Test this path hardest.

6. Managing 3 Months on Credits

Cost source	Risk	Mitigation
Gemini tokens	Medium	Use Flash not Pro; cache the system prompt
Embeddings	Low	Embed docs once, not per query
Vector DB	HIGH	Use pgvector (free) , NOT Vertex Vector Search
Agent hosting	HIGH	Run in Next.js routes , NOT Cloud Run/Agent Engine
Observability	Medium	Langfuse self-hosted , not paid tiers
Over-calling LLM	Medium	Cheap classifier first; deep-resolve only when needed

Lean monthly cost (pre-first-client)

Gemini Flash (~500 tickets/mo)	~\$5-15
Embeddings (one-time + queries)	~\$1
pgvector (Supabase – already paying)	\$0
Langfuse (self-host Railway/Fly)	~\$5
Resend inbound (free tier)	\$0
Total NEW spend	~\$10-20/month

Credit discipline: Set GCP **budget alerts at 25/50/75%** on day 1. Use Gemini Flash everywhere. Cache the system prompt (Gemini context caching = big savings). Log token cost per ticket in Langfuse so you see exactly what each resolution costs. Your 3-month credits are better spent on the heavier *collections* agent — support barely touches them.

7. The 6-Week Plan (Learn + Build Together)

Rule: never learn for more than 2 days without building the thing you learned inside DataByt.

WEEK 1 — Foundations	
Learn	Gemini tool-calling, structured output (Zod)
Build	searchKnowledge tool + embed guide docs
Ship	ask a question → get a KB answer (test script)
WEEK 2 — ADK core	
Learn	Agent, Tool, Runner, Session, State
Build	Classifier agent
Ship	ticket → classified correctly
WEEK 3 — Resolution	
Learn	ADK multi-agent (sequential), confidence
Build	Resolver agent (answer vs escalate)
Ship	70% of golden-set auto-answered
WEEK 4 — Actions	
Learn	Tool design, guardrails via callbacks
Build	Action tools (invoice, integration, resend)
Ship	agent takes a real action and replies
WEEK 5 — Human loop + intake	
Learn	Memory, escalation patterns, Resend inbound
Build	escalateToHuman + email-ticket + dashboard
Ship	email in → resolve or escalate → you notified
WEEK 6 — Reliability	
Learn	Eval, Langfuse, cost tracking
Build	golden eval set + SLA cron + budget alerts
Ship	measured 70%+ auto-resolution, fully traceable

8. What "Monopoly" Looks Like

After 6 weeks you have what competitors at your price point do not:

- A support agent that knows AR, dunning, QuickBooks, disputes cold
- Resolves 70% of tickets in <2 minutes, 24/7
- Takes real actions — not just "here's a help article"
- Escalates the hard 30% with full context
- Gets smarter with every resolved ticket (they become KB)
- Costs ~\$15/month · lets you serve 20+ clients solo before hiring

A CFO who gets an instant, correct, action-taking answer at 11pm tells other CFOs. That word-of-mouth is the moat.

*Build the support agent. Resolve 70% automatically. Stay solo until revenue says otherwise.
That is how a one-person SaaS competes with funded teams.*